

LangGraph vs LangChain: Quick Notes

LangChain is great for building linear or lightly-branching pipelines: a chain that retrieves documents, formats a prompt, and calls an LLM. It works well when the flow of control is mostly predictable.

LangGraph is built for cases where control flow needs to be dynamic: agents that loop until a task is done, agents that call other agents, or workflows with conditional branches based on the LLM's own output. It models your application as a graph of nodes (each node is a function or an LLM call) and edges (which can be conditional).

A common pattern: use LangGraph to orchestrate multiple specialized agents. For example, a Research Agent retrieves information, an Analyzer Agent extracts insights from it, and a Summarizer Agent produces the final report. LangGraph tracks the shared state as it passes between these nodes, and can route back to an earlier node if more research is needed.

Key LangGraph concepts: State is a shared object (often a TypedDict) that flows through the graph. Nodes are functions that read and update the state. Edges define which node runs next and can be conditional on state values. Checkpointing lets LangGraph save state so long-running agents can pause and resume, which is useful for human-in-the-loop workflows.